4CCS1IAI Artificial Intelligence | Guitar Planning Model

Name: Satoshi Ishida

ID: 1629145

Introduction

Artificial Intelligence is at the forefront of technological advancement among other areas of computing. We are deep in the digital age of information sharing, that allows us to manipulate and disseminate boundless amounts of data through the use of information systems.

The researcher has been tasked with producing a planning model and solutions for a chosen problem that would be useful to solve in a real-life scenario. The chosen domain is a model for playing a 24-fretted, 6-string electric guitar, in the standard "E A D G B E" tuning. This consists of playing music notes on a guitar by moving up and down a fretboard and changing strings. For simplicity, chords and guitar techniques have been excluded, however, this model can be expanded on in the future.

Whilst AI is widely used in many different prominent areas of society and is deep in research; it is also utilised in entertainment. AI has already been utilised in creating robots that perform music, and intelligent systems have been utilised in composing their own music. The following research has been carried out in-order to learn about how AI can be applied to music and to gain a good understanding of PDDL.

Domain and PDDL

The guitar planning domain is based on a guitar's fretboard that can be played by locating the music notes on each fret that belong to a separate string. A 24-fret guitar consists of 4 octaves, with each of the 6-strings having 24 frets containing music notes. These frets are on a fretboard with the strings passing over them, these are then pressed with a finger to sound a note if one strums the guitar. In real-life, an electric guitar's pickups (magnets with wire coils) transmit the current flow to an amplifier or digital software. Acoustic and classical guitars utilise their own body to sound the notes. Electro acoustic guitars are also available, which are acoustic guitars with a pickup/s for amplification.

Domain types:

- **"note"** – a musical note.
- **"fret"** – a fret on a 24-fret guitar where notes are played.
- **"string"** – a single string out of 6 to play music notes.

Domain predicates:

- **"(music-note ?n - note)"** – a music note.
- **"(fret-note ?f - fret)"** – a music note position on a fret.
- **"(guitar-string ?s - string)"** – a guitar string out of 6 in total.
- **"(linked ?n ?f - fret ?s - string)"** – a music note is linked to a fret and a string, if ?n and ?f are frets and ?s is a string, where ?n and ?f are linked to ?s. The ?n and ?f are used as "x to y positions" for moving between frets.
- **"(at-fret ?n - fret ?f - string)"** – ?n is a fret, ?f is a fret string and ?n is in ?f i.e. they are combined.

- **"(at-note-n ?n - fret ?f - string)"** – a note "n" is at a fret and a fret belongs to a string. It gets confusing, but it all makes sense in how it all works.
- **"(at-note ?f - fret)"** – ?f is a fret and a note is in ?f.

Domain functions:

- **"(move ?n - fret ?f - fret)"** – move a note on a fret from fret "X" to fret "Y". This is used to move up and down the fretboard. "?n fret" meaning a note on a fret as a default starting point and "?f fret" meaning a new fret to move to, which has a new note that is assigned to a fret. This function also calculates and returns the times taken to play music notes.

Domain durative actions:

- **"hand1-move-up-fretboard" and "hand1-move-down-fretboard"** – these are used to see whether a fret is assigned to a certain string and if it is linked with the next fret on that string. If frets are linked then the planner will move between frets until the goal is reached. Durative time is used to calculate the cost and a best possible path to be chosen by the planner.
- **"change-string"** – this durative action is used for changing guitar strings if the same fret note is on both strings, and it is used so that the model can complete an optimal search for the note that it is tasked to search for. This function works optimally in the domain; however, the researcher was not able to utilise this for the finished problem file. Due to a note having to be connected to a fret and a string, even though there are repeated notes on a guitar on different strings, they are all labelled differently in the problem file. Therefore, this action cannot be utilised and instead using the provided problem file, the planner can only search within its assigned string.

Further notes

The domain file contains a modest amount of durative actions; however, it is the problem files that are highly detailed and complicated, due to the following –

- Assigning music notes to different octaves as there are 4 octaves on a 24-fret guitar.
- Assigning each note with its octave assignment to each fret, with each string having its own 24 frets.
- Linking frets that can travel between each other and each string.

This planning model could be extended for guitars with more strings and other stringed instruments. Furthermore, extra functions could be implemented to add a higher level of complexity and to give the planning model more of a human element.

The researcher originally created a guitar domain utilising a more complicated domain model that utilised predicates and functions for moving "hand1-finger" to "hold" a "note" and "hand-2" to "pick" a note and so forth. Also, guitar techniques such as note bends and palm muting were implemented. However, when it came to testing the final PDDL files, on compile, the code was deemed unsolvable. Due to this fall-back, the researcher found a 'Route Planner' to simplify the domain. Instead, a music note was used in place of a "person" and frets instead of "locations", that are linked to one another. Moreover, instead of a user changing a travel route line, the strings would

change and this served as a great backbone for the Guitar Planner. Even though the domain is largely inspired from great past work, the scenario and problem files were created from scratch; much hard work went into converting a real-life guitar into a planning model.

Appendix

Domain

guitar-domain.PDDL guitar-problem1.PDDL

```
(define (domain guitar)
 (:requirements :typing :fluents :durative-actions :equality)
 (:types note fret string)

(:predicates
        (music-note ?n - note)
        (fret-note ?f - fret)
        (guitar-string ?s - string)
        (linked ?n ?f - fret ?s - string)
        (at-fret ?n - fret ?f - string)
        (at-note-n ?n - fret ?f - string)
        (at-note ?f - fret)
)

(:functions (move ?n - fret ?f - fret))

(:durative-action hand1-move-up-fretboard
        :parameters (?n ?f - fret ?s - string)
        :duration   (= ?duration (move ?n ?f))
        :condition  (and (at start (guitar-string ?s))
                    (at start (at-note-n ?n ?s))
                    (at start (at-note ?n))
                    (over all(linked ?n ?f ?s)))
        :effect     (and (at end (at-note ?f))
                    (at start (not (at-note ?n)))
                    (at end (at-note-n ?f ?s))
                    (at start (not (at-note-n ?n ?s))))
)

(:durative-action hand1-move-down-fretboard
        :parameters (?n ?f - fret ?s - string)
        :duration   (= ?duration (move ?f ?n))
        :condition  (and (at start (guitar-string ?s))
                    (at start (at-note-n ?n ?s))
                    (at start (at-note ?n))
                    (over all(linked ?f ?n ?s)))
        :effect     (and (at end (at-note ?f))
                    (at start (not (at-note ?n)))
                    (at end (at-note-n ?f ?s))
                    (at start (not (at-note-n ?n ?s))))
)

(:durative-action change-string
        :parameters (?n - fret ?f ?s - string)
        :duration   (= ?duration 0.5)
        :condition  (and (at start (at-fret ?n ?f))
                    (at start (at-fret ?n ?s))
                    (at start (at-note ?n))
                    (at start (at-note-n ?n ?f)))
        :effect     (and (at end (guitar-string ?s))
                    (at start (not (at-note-n ?n ?f)))
                    (at end (at-note-n ?n ?s))))
)
```